

Microservices Pattern — Complete Guide & C# Implementation

Ali Elkomy | Senior Full Stack Developer | .NET 8 + MassTransit + RabbitMQ

Table of Contents

1. What is Microservices?
 2. Famous Real-World Examples
 3. Our Solution Architecture
 4. Problem: Race Condition (Two Users, Last Item)
 5. Problem: Order Placed but Not Invoiced
 6. Outbox Pattern
 7. Idempotency Key
 8. MassTransit Saga State Machine
 9. Dead Letter Queue
 10. Solution Architecture Summary
 11. C# Solution Structure
-

1. What is Microservices?

Microservices is an **architectural style** where a large application is broken into **small, independent services**, each responsible for a single business capability.

Each service:

- Runs in its own **process**
- Has its own **database**
- Communicates over **lightweight protocols** (HTTP/REST, gRPC, or messaging queues)
- Can be **deployed independently**

Core Principles

Principle	Description
Single Responsibility	One service = one business domain
Loose Coupling	Services don't know each other's internals
High Cohesion	Related logic lives together
Independent Deployability	Deploy one service without touching others
Decentralized Data	Each service owns its data store

Microservices vs Monolith

	Monolith	Microservices
Deployment	All or nothing	Per service
Scaling	Scale everything	Scale only what needs it
DB	One shared DB	One DB per service
Failure	One crash = all down	Isolated failures
Team	One team owns all	Teams own services

2. Famous Real-World Examples

Netflix — hundreds of microservices: Auth, Streaming, Recommendation, Billing — each independently scaled.

Amazon — product page, cart, payment, recommendation are all separate services. Jeff Bezos mandated service boundaries internally ("the Bezos API Mandate").

Uber — Trip Service, Driver Service, Pricing Service, Notification Service, Maps Service — all independent.

Spotify — Squad-based model: Playlist Service, Search Service, Player Service — each team owns their service.

3. Our Solution Architecture

Scenario: E-Commerce Platform

Services:

Service	Technology	Responsibility
API Gateway	YARP / Ocelot	Single entry point, route requests
Product Service	ASP.NET Core + EF Core	Manage product catalog
Order Service	ASP.NET Core + EF Core + MassTransit	Create orders, publish events, host Saga
Invoice Service	ASP.NET Core + EF Core + MassTransit	Create invoices on OrderPlaced event
Notification Service	.NET Worker + MassTransit	Send emails on events
Shared.Contracts	Class Library	DTOs, event contracts, shared models

Communication:

- **Sync:** REST via API Gateway → Services
- **Async:** Order placed → RabbitMQ event → Invoice + Notification react

4. Problem: Race Condition

Scenario

Two users request the last item in stock at the same millisecond.

What goes wrong (without protection)

```

T=0ms  User A: READ qty=1  → OK, proceed
T=0ms  User B: READ qty=1  → OK, proceed (before A commits)
T=5ms  User A: WRITE qty=0 → success
T=5ms  User B: WRITE qty=-1 → OVERSELL!

```

text

This is a **Lost Update** — one transaction overwrites the other without knowing it happened.

Solutions

Solution 1: Optimistic Concurrency (EF Core RowVersion)

Add a `[Timestamp]` byte array to the entity. EF Core adds `WHERE RowVersion = @original` on every UPDATE. If the row changed, it throws `DbUpdateConcurrencyException`.

```
public class Product
{
    public Guid Id { get; set; }
    public string Name { get; set; }
    public int Quantity { get; set; }

    [Timestamp]
    public byte[] RowVersion { get; set; } // EF Core manages this
}

// On update – EF Core auto-generates:
// UPDATE Products SET Qty=@new WHERE Id=@id AND RowVersion=@original
// If 0 rows affected → throw DbUpdateConcurrencyException
```

csharp

Best for: Low-contention. User B gets an error and retries.

Solution 2: Pessimistic Locking

Lock the row when reading so no one else can touch it.

```
BEGIN TRANSACTION;
SELECT * FROM Products WITH (UPDLOCK) WHERE Id = @id;
UPDATE Products SET Qty = Qty - 1 WHERE Id = @id;
COMMIT;
```

sql

Best for: High-contention, critical inventory. Slower throughput.

Solution 3: Atomic UPDATE — Recommended

Skip the READ entirely. Decrement and guard in one SQL statement.

csharp

```
var updated = await db.Database.ExecuteSqlRawAsync(
    "UPDATE Products SET Qty = Qty - 1 WHERE Id = {0} AND Qty > 0",
    productId);

if (updated == 0)
    throw new OutOfStockException("Product is out of stock");
```

Best for: Any scenario. No race condition possible — DB serializes atomically.

Solution 4: Distributed Lock with Redis

For microservices running across multiple servers.

csharp

```
var lockKey = $"product-lock:{productId}";
var acquired = await redis.StringSetAsync(
    lockKey, "locked",
    expiry: TimeSpan.FromSeconds(5),
    when: When.NotExists); // SETNX

if (!acquired)
    throw new ConflictException("Retry in a moment");

try {
    await DecrementAndOrder(productId);
}
finally {
    await redis.KeyDeleteAsync(lockKey); // always release
}
```

Recommendation by Scenario

Scenario	Best Solution
Single DB, low traffic	Optimistic Concurrency (RowVersion)
Single DB, high traffic	Atomic UPDATE WHERE qty > 0
Microservices, distributed	Redis Lock + Atomic UPDATE
Financial / critical	Pessimistic Lock + Saga

5. Problem: Order Placed but Not Invoiced

The Distributed Transaction Problem

In a monolith: one DB, one `COMMIT` / `ROLLBACK` — atomic by default.

In microservices: each service has its own DB. If Order Service saves but Invoice Service crashes:

-  Order exists in Orders DB
-  Qty decremented in Products DB
-  Invoice never created
-  Payment unclear

The system is in an inconsistent state with no automatic rollback.

Solution: SAGA Pattern

A Saga is a **sequence of local transactions** where each step publishes an event triggering the next. If any step fails, **compensating transactions** undo previous steps in reverse.

Happy Path

Order Saved (PENDING)

- Inventory Reserved
- Payment Processed
- Invoice Created
- Order CONFIRMED 

text

Failure Path (Invoice fails)

Invoice Failed

- Refund Payment (compensating)
- Release Inventory (compensating)
- Cancel Order (compensating)
- Notify User (compensating)

text

Two Saga Styles

Style	How	Best For
Choreography	Each service reacts to events	Simple flows, loose coupling
Orchestration	Central coordinator directs services	Complex flows, easier to debug

6. Outbox Pattern

Problem it solves

Order Service saves order to DB **then crashes before publishing the event** — event is lost forever.

How it works

Save the order AND the outbox message in **one atomic transaction**. A background worker polls and publishes.

OrderPlacedEvent (Shared.Contracts)

```

public class OrderPlacedEvent
{
    public Guid    EventId    { get; set; }    // idempotency key – set once,
travels everywhere
    public Guid    OrderId    { get; set; }
    public Guid    ProductId  { get; set; }
    public Guid    UserId     { get; set; }
    public int     Quantity   { get; set; }
    public DateTime PlacedAt  { get; set; }
}

```

csharp

OutboxMessage entity (Order Service DB)

csharp

```
public class OutboxMessage
{
    public Guid Id { get; set; }
    public string Type { get; set; } // event class name
    public string Payload { get; set; } // JSON serialized event
    public DateTime CreatedAt { get; set; }
    public bool Processed { get; set; }
}
```


CreateOrder — atomic write

```

public async Task<IActionResult> CreateOrder([FromBody] CreateOrderDto dto)
{
    var order = new Order
    {
        Id          = Guid.NewGuid(),
        ProductId   = dto.ProductId,
        UserId      = dto.UserId,
        Quantity    = dto.Quantity,
        Status      = OrderStatus.Pending,
        CreatedAt   = DateTime.UtcNow
    };

    var orderEvent = new OrderPlacedEvent
    {
        EventId     = Guid.NewGuid(),    // ← you create EventId here
        OrderId     = order.Id,
        ProductId   = order.ProductId,
        UserId      = order.UserId,
        Quantity    = order.Quantity,
        PlacedAt    = order.CreatedAt
    };

    await using var tx = await db.Database.BeginTransactionAsync();

    db.Orders.Add(order);
    db.OutboxMessages.Add(new OutboxMessage
    {
        Id          = Guid.NewGuid(),
        Type        = nameof(OrderPlacedEvent),
        Payload     = JsonSerializer.Serialize(orderEvent),
        CreatedAt   = DateTime.UtcNow,
        Processed   = false
    });

    await db.SaveChangesAsync();
    await tx.CommitAsync();

    return Ok(order.Id);
}

```

OutboxWorker (Background Service in Order Service)

csharp

```
public class OutboxWorker : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            var pending = await db.OutboxMessages
                .Where(m => !m.Processed)
                .ToListAsync(ct);

            foreach (var msg in pending)
            {
                var @event = JsonSerializer.Deserialize<OrderPlacedEvent>
(msg.Payload);
                await publishEndpoint.Publish(@event, ct);
                msg.Processed = true;
            }

            await db.SaveChangesAsync(ct);
            await Task.Delay(5000, ct);
        }
    }
}
```

7. Idempotency Key

Problem it solves

If RabbitMQ redelivers a message (network glitch, retry), the Invoice Service might create **two invoices** for the same order.

How it works

`EventId` is set **once** in Order Service when creating `OrderPlacedEvent`. It travels inside the message payload. Each consumer checks if it already processed that `EventId`.

ProcessedEvent entity (Invoice Service DB)

csharp

```
public class ProcessedEvent
{
    public Guid      EventId      { get; set; }    // PK + unique index
    public DateTime ProcessedAt { get; set; }
}
```

InvoiceConsumer

csharp

```
public class InvoiceConsumer : IConsumer<OrderPlacedEvent>
{
    public async Task Consume(ConsumeContext<OrderPlacedEvent> context)
    {
        var msg = context.Message; // msg.EventId = the Guid set in Order
        Service

        var duplicate = await db.ProcessedEvents
            .AnyAsync(e => e.EventId == msg.EventId);

        if (duplicate) return; // already processed – safe to ignore

        var invoice = new Invoice
        {
            Id          = Guid.NewGuid(),
            OrderId     = msg.OrderId,
            UserId      = msg.UserId,
            Amount      = await CalculateAmount(msg.ProductId, msg.Quantity),
            CreatedAt   = DateTime.UtcNow
        };

        db.Invoices.Add(invoice);
        db.ProcessedEvents.Add(new ProcessedEvent
        {
            EventId      = msg.EventId,
            ProcessedAt  = DateTime.UtcNow
        });

        await db.SaveChangesAsync(); // atomic – both or neither
    }
}
```

8. MassTransit Saga State Machine

What is built-in vs what you write

Item	Source
<code>MassTransitStateMachine<T></code>	✓ MassTransit NuGet — base class you inherit
State	✓ MassTransit NuGet — type for each state
<code>Event<T></code>	✓ MassTransit NuGet — type for each trigger
<code>Initially(...)</code>	✓ MassTransit DSL — "when starting fresh"
<code>During(...)</code>	✓ MassTransit DSL — "when already in this state"
<code>When(...)</code>	✓ MassTransit DSL — "if this event arrives"
<code>.TransitionTo(...)</code>	✓ MassTransit DSL — move to new state
<code>.Publish(...)</code>	✓ MassTransit DSL — send a message
<code>.Then(...)</code>	✓ MassTransit DSL — run custom code
State property names	✎ You define
Event property names	✎ You define
Transition logic	✎ You define
Message classes	✎ You define

OrderState (DB row that tracks one saga instance)

```

public class OrderState : SagaStateMachineInstance
{
    public Guid CorrelationId { get; set; } // = OrderId
    public string CurrentState { get; set; } // MassTransit writes state
    name
    public Guid UserId { get; set; }
    public Guid ProductId { get; set; }
    public int Quantity { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

csharp

OrderStateMachine

csharp

```

public class OrderStateMachine : MassTransitStateMachine<OrderState>
{
    public State Pending { get; private set; }
    public State Invoiced { get; private set; }
    public State Cancelled { get; private set; }

    public Event<OrderPlacedEvent> OrderPlaced { get; private set; }
    public Event<InvoiceCreatedEvent> InvoiceCreated { get; private set; }
    public Event<InvoiceFailedEvent> InvoiceFailed { get; private set; }

    public OrderStateMachine()
    {
        InstanceState(x => x.CurrentState);

        Event(() => OrderPlaced,
            x => x.CorrelateById(ctx => ctx.Message.OrderId));
        Event(() => InvoiceCreated,
            x => x.CorrelateById(ctx => ctx.Message.OrderId));
        Event(() => InvoiceFailed,
            x => x.CorrelateById(ctx => ctx.Message.OrderId));

        Initially(
            When(OrderPlaced)
                .Then(ctx => {
                    ctx.Saga.UserId = ctx.Message.UserId;
                    ctx.Saga.ProductId = ctx.Message.ProductId;
                    ctx.Saga.Quantity = ctx.Message.Quantity;
                    ctx.Saga.CreatedAt = DateTime.UtcNow;
                })
                .TransitionTo(Pending)
                .Publish(ctx => new CreateInvoiceCommand {
                    OrderId = ctx.Saga.CorrelationId,
                    UserId = ctx.Saga.UserId,
                    ProductId = ctx.Saga.ProductId,
                    Quantity = ctx.Saga.Quantity
                })
        );

        During(Pending,
            When(InvoiceCreated)
                .TransitionTo(Invoiced),

            When(InvoiceFailed)
                .TransitionTo(Cancelled)
                .Publish(ctx => new ReleaseInventoryCommand {

```

```
        OrderId = ctx.Saga.CorrelationId })
    .Publish(ctx => new NotifyUserCommand {
        UserId = ctx.Saga.UserId,
        Reason = "Invoice failed – order cancelled" })
    );
}
```

9. Dead Letter Queue

What it solves

Messages that fail all retries are moved to a Dead Letter Queue instead of being silently dropped.

Where to configure

Every consumer service — in its own `Program.cs`.

Flow:

RabbitMQ: invoice-queue

- attempt 1 fails → wait 1s
- attempt 2 fails → wait 3s
- attempt 3 fails → wait 5s
- all retries exhausted
- message moved to: invoice-queue_error (DLQ)
- ops team inspects and replays

text

Configuration (Invoice Service Program.cs)

csharp

```
builder.Services.AddMassTransit(x =>
{
    x.AddConsumer<InvoiceConsumer>();
    x.AddConsumer<NotificationConsumer>();

    x.UsingRabbitMq((ctx, cfg) =>
    {
        cfg.Host("rabbitmq://localhost");

        cfg.UseMessageRetry(r =>
            r.Intervals(
                TimeSpan.FromSeconds(1),
                TimeSpan.FromSeconds(3),
                TimeSpan.FromSeconds(5)
            ));

        cfg.ReceiveEndpoint("invoice-queue", e => {
            e.ConfigureConsumer<InvoiceConsumer>(ctx);
            e.DeadLetterExchange = "dead-letter-exchange";
        });

        cfg.ReceiveEndpoint("notification-queue", e => {
            e.ConfigureConsumer<NotificationConsumer>(ctx);
            e.DeadLetterExchange = "dead-letter-exchange";
        });
    });
});
```

10. Solution Summary

Where each piece lives

Pattern	Lives in	Configured in
Outbox table + Worker	Order Service	OrderDbContext + Program.cs
OrderPlacedEvent class	Shared.Contracts	referenced by all
ProcessedEvent table	Invoice Service	InvoiceDbContext

Pattern	Lives in	Configured in
Saga State Machine	Order Service	Program.cs
DLQ retry config	Every consumer service	each Program.cs
Redis Lock	Order Service	Program.cs

Defense in Depth

Order Placed

[text](#)

- Atomic UPDATE prevents oversell
- Outbox guarantees event is published
- Saga tracks state across all services
- Idempotency prevents duplicate processing
- DLQ catches anything that still fails
- Compensating transactions undo partial work

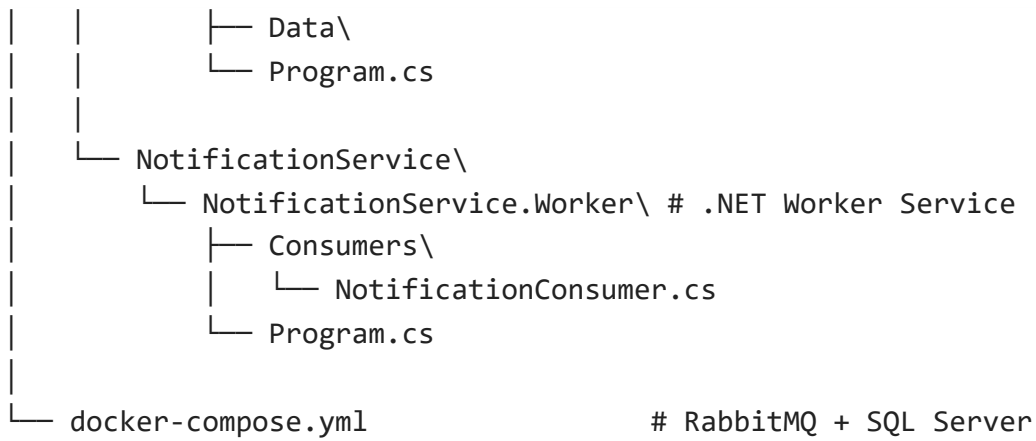
11. C# Solution Structure

text

```

D:\Ali\Microservice\
|
|— ECommerce.sln
|
|— src\
|   |
|   |— Shared\
|   |   |
|   |   |— Shared.Contracts\           # DTOs, Events, Commands
|   |   |   |
|   |   |   |— Events\
|   |   |   |   |
|   |   |   |   |— OrderPlacedEvent.cs
|   |   |   |   |— InvoiceCreatedEvent.cs
|   |   |   |   |— InvoiceFailedEvent.cs
|   |   |   |— Commands\
|   |   |   |   |
|   |   |   |   |— CreateInvoiceCommand.cs
|   |   |   |   |— ReleaseInventoryCommand.cs
|   |   |   |   |— NotifyUserCommand.cs
|   |
|   |— ApiGateway\
|   |   |
|   |   |— ApiGateway\           # YARP reverse proxy
|   |   |   |
|   |   |   |— Program.cs
|   |   |   |— appsettings.json    # route config
|   |
|   |— ProductService\
|   |   |
|   |   |— ProductService.Api\      # ASP.NET Core Web API
|   |   |   |
|   |   |   |— Controllers\
|   |   |   |— Entities\
|   |   |   |— Data\
|   |   |   |— Program.cs
|   |
|   |— OrderService\
|   |   |
|   |   |— OrderService.Api\        # ASP.NET Core Web API + Saga host
|   |   |   |
|   |   |   |— Controllers\
|   |   |   |— Entities\
|   |   |   |— Data\
|   |   |   |— Sagas\
|   |   |   |   |
|   |   |   |   |— OrderState.cs
|   |   |   |   |— OrderStateMachine.cs
|   |   |   |— Workers\
|   |   |   |   |
|   |   |   |   |— OutboxWorker.cs
|   |   |   |— Program.cs
|   |
|   |— InvoiceService\
|   |   |
|   |   |— InvoiceService.Api\      # ASP.NET Core Web API
|   |   |   |
|   |   |   |— Consumers\
|   |   |   |   |
|   |   |   |   |— InvoiceConsumer.cs
|   |   |   |— Entities\

```



NuGet Packages per service

Order Service:

- `MassTransit.RabbitMQ`
- `MassTransit.EntityFrameworkCore`
- `Microsoft.EntityFrameworkCore.SqlServer`

Invoice Service:

- `MassTransit.RabbitMQ`
- `Microsoft.EntityFrameworkCore.SqlServer`

Notification Service:

- `MassTransit.RabbitMQ`

API Gateway:

- `Yarp.ReverseProxy`

Document generated from live architecture session — Ali Elkomy, June 2026